

测试篇期末复习

依据：软件测试课件，123.txt 答疑转写，考试题目类型与重点，2020/2022 往年卷测试题。

使用建议

测试题主要考“会不会选测试用例”。不要只背概念，要能从规格或代码中找边界、条件、路径，并写成清晰的测试用例表。

一、考试定位与优先级

优先级	内容	依据	答题产物
高	黑盒测试：等价类划分、边界值分析	2020 测试题直接考边界值；课件重点	输入值、预期输出、覆盖理由
高	白盒测试：语句覆盖、条件覆盖、路径覆盖	2022 测试题考条件覆盖；课件重点	覆盖目标 + 最小测试用例集
中高	圈复杂度	2022 测试题和构造篇均涉及	判定节点数 + 1，或控制流图 E-N+2
中	测试级别：单元、集成、系统、验收、回归	课件基本概念；大作业链路也会涉及	每一级测什么、依据什么、是否需要桩/驱动
中	测试用例选择与 AI 辅助测试	课程新版内容	AI 可生成，但人要审查边界、约束和检错能力
低	测试相关图	答疑说图相关不用考虑	不作为本篇重点

考试提醒

答疑中“测试相关的图不用考虑吧”的回复意味着测试篇不必把状态图、协作图等图形测试方法作为核心。更应该把时间放在边界值、等价类、条件覆盖、路径覆盖这些能落到测试用例的内容上。

二、软件测试基础

1. 软件测试的目标

- 测试是为了发现缺陷，而不是证明程序没有缺陷。
- 测试验证两件事：是否在构建正确的东西，是否把东西构建正确。
- 测试用例通常包括：输入、执行条件、测试步骤、预期结果，必要时还包括实际结果和通过/失败判定。
- 好的测试用例要有明确目的，能覆盖风险高、容易出错或需求关键的场景。

测试技术

- 测试用例的选择
- 随机测试
- 基于规格的技术 - 黑盒测试方法
- 基于代码的技术 - 白盒测试方法
- 特定测试技术

课件截图：测试技术。考试主要落在黑盒测试方法和白盒测试方法。

2. 测试级别

级别	关注点	依据/对象	是否常用桩和驱动
单元测试	单个函数、类、模块是否正确	详细设计、代码、接口契约	常用桩和驱动隔离依赖
集成测试	模块之间接口、调用顺序、数据传递是否正确	体系结构设计、接口设计、协作关系	常用桩和驱动；自顶向下/自底向上
系统测试	整个系统是否满足需求	需求规格、用例文档、非功能需求	通常不依赖桩和驱动
验收测试	用户是否认可系统满足业务目标	业务需求、验收标准、真实场景	由用户或代表参与
回归测试	修改后原有功能是否被破坏	已有有效测试用例集、变更影响范围	自动化很重要

系统测试

- 单元测试、集成测试更加关注技术上的正确性，重在发现设计缺陷和代码缺陷。系统测试则不同，它更关注不符合需求的缺陷和需求自身的内在缺陷。
- 根据测试目标的不同，有很多不同类型的系统测试：功能测试、非功能性测试、验收测试、安装测试等等。但是发生在软件测试阶段，完全由软件测试人员控制和执行的主要是功能测试和非功能性测试。
- 系统测试关注整个系统的行为，所以不依赖于桩程序和驱动程序。但是，使用一些测试工具可以让系统测试过程更加自动化。
- 系统测试的功能测试计划以需求规格说明文档或用例文档为基础，主要使用随机测试和基于规格的测试技术设计功能测试用例。在测试非功能性需求时需要使用针对非功能需求的特定测试技术进行测试计划和测试用例设计。

课件截图：系统测试。系统测试以需求规格说明或用例文档为基础。

三、黑盒测试

黑盒测试基于规格说明设计测试用例，不关心程序内部代码结构。考试中看到业务规则、输入范围、输出范围时，优先想到黑盒测试。

1. 等价类划分

- 把输入域划分为若干等价类，认为同一类中的数据对程序行为具有相同代表性。
- 有效等价类：符合规格说明、合理有意义的数据集合。
- 无效等价类：不符合规格说明、非法或异常的数据集合。
- 设计用例时至少覆盖每个有效等价类，并尽量让每个无效等价类单独出现，便于定位原因。

黑盒测试方法 - 等价类划分

有效等价类：是指对于程序的规格说明来合理、意义输入数据构成集合。利用有效等价类可检验程序是否实现了规格说明中所定的功能和性能。

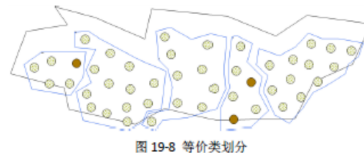


图 19-8 等价类划分

课件截图：等价类划分。有效等价类用于检验程序是否实现规格说明中的功能和性能。

2. 边界值分析

- 边界值分析关注输入或输出等价类的边界及边界附近，因为错误最容易发生在边界。
- 常用取值：最小值、最小值+1、最大值-1、最大值；必要时加入刚低于最小值、刚高于最大值。
- 如果规则分段，例如 3、7、10、15、20 天，就在每个分界点附近取值。
- 如果题目规格有空洞或歧义，应在答案中指出，并设计测试用例暴露该问题。

黑盒测试方法

- 边界值分析

表 19-4 Sales 类 getChange(double payment) 方法的边界值分析测试用例

ID	输入		预期输出
1	payment=50	total=50	0
2	payment=50	total=49	1
3	payment=50	total=51	输入数据无效
4	payment=1	total=1	0
5	payment=0	total=0	0
6	payment=-1	total=10	输入数据无效

课件截图：边界值分析。2020 测试题正是这种类型。

3. 决策表与状态转换

- 决策表适合多个条件组合决定多个动作的业务规则，例如优惠、权限、审批。
- 状态转换测试适合对象有明确状态和事件，例如订单、借阅、账户冻结/解冻。
- 本次复习中图相关不是重点，但要知道它们都属于基于规格的黑盒测试方法。

四、白盒测试

白盒测试基于代码结构设计测试用例，关注语句、分支、条件、路径是否被执行。考试中看到代码片段、if/while/... 时，优先想到白盒覆盖。

1. 常见覆盖准则

覆盖准则	含义	答题抓手
语句覆盖	每条语句至少执行一次	找一组能让所有语句跑到的输入
判定/分支覆盖	每个判定的真、假分支至少执行一次	每个 if/while 至少 T/F 各一次
条件覆盖	每个判定中每个原子条件的真、假至少满足一次	拆出 C1、C2、C3；注意短路求值
路径覆盖	每条独立执行路径至少执行一次	根据控制流图或分支组合列路径

白盒测试方法 - 语句覆盖

语句覆盖设计测试用例的标准是确保被测试对象的每一行程序代码都至少执行一次。

```
public Class Customer {  
    int bonus;//积分额度  
    .....  
    //预计算消费行为后的积分额  
    int getBonus ( boolean cashPayment , int consumption , bo  
    //已有的 bonus 需要调整 getBonus，不能直接使用  
    int preBonus=this.getBonus();  
    if (cashPayment) {  
        preBonus+=consumption;  
    }  
    if (vip) {  
        preBonus*=1.5;  
    } else {  
        preBonus*=1.2;  
    }  
    return preBonus;  
}
```

课件截图：语句覆盖。语句覆盖最弱，只要求每行代码至少执行一次。

白盒测试方法

- 条件覆盖

条件覆盖设计测试用例的标准是确保程序中每个判断的每个结果都至少满足一次

表 19-7 Customer.getBonus() 的条件覆盖测试用例

ID	输入				预期输出
1	getBonus ()=100	cashPayment=true	consumption=100	vip=true	300
2	getBonus ()=100	cashPayment=false	consumption=100	vip=false	120

课件截图：条件覆盖。2022 测试题的关键词就是条件覆盖。

白盒测试方法

- 路径覆盖

路径覆盖测试用例的标准是确保程序中每条独立执行路径都至少执行一次

表 19-8 Customer.getBonus() 的路径覆盖测试用例

ID	输入				预期输出
1	getBonus ()=100	cashPayment=true	consumption=100	vip=true	300
2	getBonus ()=100	cashPayment=false	consumption=100	vip=false	120
	getBonus ()=100	cashPayment=true	consumption=100	vip=false	240
	getBonus ()=100	cashPayment=false	consumption=100	vip=true	150

课件截图：路径覆盖。路径覆盖更强，但路径数量可能很快膨胀。

2. 短路求值

- `A && B && C` 中，如果 A 为假，B 和 C 不会被求值。
- `A || B || C` 中，如果 A 为真，B 和 C 不会被求值。
- 做条件覆盖时，不能只让某个条件在逻辑上可能为真/假，还要确保程序实际求值到该条件。
- 因此为了让后面的条件出现假值，前面的条件必须先被设计为真。

3. 圈复杂度

- 快速算法：圈复杂度 $V(G) = \text{判定节点数} + 1$ 。

- 判定节点通常包括 if、else if、while、for、case、catch，以及复杂布尔表达式中独立判断的拆分口径。
- 控制流图算法： $V(G) = E - N + 2P$ 。单个连通程序片段中 $P=1$ 。
- 考试常用结论：圈复杂度越高，独立路径越多，测试和维护成本越高。

五、基于用例的系统测试

系统测试的功能测试计划可以以需求规格说明或用例文档为基础。基于用例的测试不是让你重画用例图，而是从用例主成功场景、扩展场景和异常场景中提取测试用例。

来源	测试关注点	示例
用例前置条件	条件不满足时系统是否拒绝或提示	未登录时不能充值
主成功场景	正常流程能否完成	选择充值 - 输入金额 - 支付成功 - 更新余额
扩展场景	可选流程、分支流程是否正确	选择不同支付方式、取消支付
异常场景	错误输入和外部失败是否处理	金额为负、支付失败、网络超时
后置条件	状态和数据是否一致	余额增加、交易记录生成、失败不扣款

和需求篇的关系

需求题要求你会写用例；测试题则要求你能把用例转换成测试。一个完整用例至少可以派生正常、边界、异常、权限、状态一致性等测试场景。

六、AI 辅助测试的答题口径

- AI 可以辅助生成测试框架、等价类、边界值、决策表、状态转换测试用例，也可以根据代码生成白盒覆盖用例。
- AI 输出不能直接相信，需要人工检查：是否遗漏业务约束、是否覆盖边界、预期结果是否正确、是否只追求覆盖率而检错能力弱。
- 高风险模块、支付/安全/隐私等关键逻辑应保留人工审查和关键测试用例。
- 可以用覆盖率报告、历史缺陷、代码变更范围来让 AI 帮助补充回归测试。

大模型辅助白盒测试与覆盖率提升

大模型辅助控制流分析

- 输入代码，大模型自动识别独立路径，生成路径覆盖测试用例
- 分析圈复杂度，对高复杂度方法生成分支覆盖和条件覆盖用例

大模型辅助提升覆盖率

- 将覆盖率报告（JaCoCo）输入大模型，自动识别未覆盖分支并生成补充用例
- Prompt：'以下方法的测试覆盖率为72%，未覆盖的分支是X，请生成能覆盖这些分支的测试用例'

大模型辅助数据流测试

- 识别变量的定义-使用链（def-use pairs），生成数据流测试用例
- 检测变量未初始化使用、重复赋值未使用等潜在问题

工具集成

- JaCoCo（覆盖率） + GitHub Copilot（用例生成） + Diffblue Cover（自动化白盒测试）
- 工作流：运行测试 → 导出覆盖率报告 → 大模型分析缺口 → 生成补充测试 → 再次运行

课件截图：大模型辅助白盒测试与覆盖率提升。重点是辅助，不是替代工程判断。

七、例题与参考答案

例题 1：2020 仓库订货规则边界值测试

题目核心：供应商交货期天数为输入，备货销量天数为返回值。规则分界点为 3、7、10、15、20 天，要求用边界值法设计测试。

测试编号	输入交货期	预期结果	覆盖理由
T1	2	7 天销量	低于 3 的边界附近
T2	3	7 天销量	3 天以内的边界值，按“以内”理解为 ≤ 3
T3	4	15 天销量	刚超过 3，进入 7 天以内区间
T4	6	15 天销量	低于 7 的边界附近
T5	7	15 天销量	7 天以内的边界值
T6	8	20 天销量	刚超过 7，进入 10 天以内区间
T7	9	20 天销量	低于 10 的边界附近
T8	10	20 天销量	10 天以内的边界值
T9	11	30 天销量	刚超过 10，进入 15 天以内区间
T10	14	30 天销量	低于 15 的边界附近
T11	15	30 天销量	15 天以内的边界值
T12	16	需求需澄清/应有明确规则	15 到 20 之间存在规格空洞
T13	19	需求需澄清/应有明确规则	刚低于 20，继续暴露规格空洞
T14	20	40 天销量	20 天以上的边界值，按“以上”理解为 ≥ 20
T15	21	40 天销量	刚超过 20

答题加分点

如果题目只要求设计测试用例，可以给出边界表；如果允许评价需求，应指出 $15 < \text{交货期} < 20$ 的规则没有定义，真实项目中必须澄清需求，否则无法判断预期输出。

例题 2：2022 条件覆盖和圈复杂度

```
if (new1.indexOf(new2) != -1
    && old.indexOf(new1) == -1
    && new2.indexOf(new1) != -1
    && new1.indexOf(old) == -1) {
    if (dao.loginUser(username, old)) {
        dao.rePassword(new1);
        writer.print("yes");
    } else {
        writer.print("no");
    }
} else {
    writer.print("no");
}
```

1. 圈复杂度

若按两个 if 判定计算，圈复杂度 = 2 + 1 = 3。若按复杂布尔表达式中的每个 `&&` 条件拆分为独立判定，则外层 4 个原子条件 + 内层 loginUser 共 5 个判定，圈复杂度 = 5 + 1 = 6。考试中如果题目强调短路条件覆盖，建议说明采用拆分原子条件的口径。

2. 100% 条件覆盖测试用例

令 C1: new1 包含 new2；C2: old 不包含 new1；C3: new2 包含 new1；C4: new1 不包含 old。由于 `&&` 短路，若要评价后面的条件，前面的条件必须为真。

用例	old	new1	new2	实际求值结果	覆盖目的
T1	x	abc	z	C1=F	覆盖 C1 假；后续不求值
T2	xxabcxx	abc	a	C1=T, C2=F	覆盖 C2 假
T3	x	abc	a	C1=T, C2=T, C3=F	覆盖 C3 假
T4	a	abc	abc	C1=T, C2=T, C3=T, C4=F	覆盖 C4 假
T5	x	abc	abc	C1=T, C2=T, C3=T, C4=T	覆盖 C1-C4 真，并进入内层 if

为什么至少要这么多

在 `&&` 短路下，要让第 k 个条件取假，前 k-1 个条件必须先取真，所以 C1 假、C2 假、C3 假、C4 假和全真通常需要分开构造。

八、答题模板

1. 黑盒测试用例设计模板

步骤	写法
找输入/输出	输入是什么，输出或系统状态是什么
划等价类	有效类、无效类分别列出
找边界	每个区间边界取边界值和边界附近值
写表格	测试编号、输入、预期输出、覆盖理由
指出歧义	若规格有空洞、重叠或含糊，明确写“需求需澄清”

2. 白盒覆盖题模板

步骤	写法
拆条件	把 if 中的原子条件标为 C1、C2、C3
看短路	`&&` 后面的条件要在前面为真时才求值；`  ` 后面的条件要在前面为假时才求值
列覆盖目标	每个条件 T/F；或每条语句、每条路径
构造最小集	一行一个测试用例，写输入值和覆盖的条件结果
算复杂度	判定节点 + 1；说明是否拆分复合条件

3. 概念速记

概念	速记答案
黑盒测试	基于规格，不看内部代码，典型方法有等价类、边界值、决策表、状态转换
白盒测试	基于代码结构，看内部控制流，典型方法有语句覆盖、分支覆盖、条件覆盖、路径覆盖
边界值分析	在等价类边界及边界附近选测试用例，因为边界最容易出错
条件覆盖	每个判定中每个原子条件的真、假都至少满足一次，注意短路求值
路径覆盖	每条独立执行路径至少执行一次，覆盖强但成本高
回归测试	修改后重新运行相关测试，确认原有功能没有被破坏